



Electric Bus Location Tracker System

Database Design Document

V 2.0

By

- | | | |
|---|-----------------|------------------|
| 1 | Mehroz Ali Khan | NUM-BSCS-2024-34 |
| 2 | Ali Abbas | NUM-BSCS-2024-06 |
| 3 | Laiba Taj | NUM-BSCS-2024-31 |

Department of Computer Sciences

Namal University
Mianwali, Pakistan

Submission Date: 14th June, 2026



REVISION HISTORY

Date	Version	Description	Approved by
21/06/26	V 2.0	Final database implementation: relational schema, normalization, DDL/DML scripts, GUI, stored procedures, triggers, and GUI integration documented.	—
15/05/26	V 1.0	Initial ERD and conceptual database design (Milestone 2).	—



CONTENTS

Revision History	1
1 Chapter 1: Project Overview	3
1.1 Introduction	3
1.2 Problem Statement	3
1.3 Project Objectives	3
2 Chapter 2: Conceptual Database Design	4
2.1 Entity	4
2.2 Data Dictionary	5
2.3 Business Rules	13
2.4 Entity Relationship Diagram	14
3 Chapter 3: Logical Database Design	14
3.1 Relational Schema	14
3.2 Functional Dependencies	15
3.3 Normalization	17
4 Chapter 4: Implementation	19
4.1 Language/Framework	19
4.2 Database Connectivity	20
4.3 Stored Procedures and Functions	20
4.4 Interfaces	23
5 Task Division	24
6 References	24
7 Appendix: AI Usage and Prompts	25

CHAPTER 1: PROJECT OVERVIEW

1.1. INTRODUCTION

The Electric Bus Location Tracker System is a mobile-based application developed for the electric bus transportation network operating in Mianwali, Punjab, Pakistan, under the Punjab Mass Transit Authority (PMA). The system is designed to provide real-time tracking of electric buses, manage bus routes and stops, schedule driver duties, and generate operational reports for administrators.

The platform serves three categories of stakeholders: passengers, who use the public-facing screens to search routes, view live bus locations, and check estimated arrival times; drivers, who view their assigned duties, start and complete duty cycles, and share their live GPS location while on duty; and administrators, who manage drivers, buses, routes, stops, schedules, duty assignments, and operational reports.

The database forms the backbone of the entire system. It stores user accounts, driver and administrator profiles, bus and route information, stop locations, trip schedules, duty assignments, live GPS location history, generated reports, and a complete audit trail of system activity. This document describes the conceptual, logical, and physical design of that database, along with its implementation through DDL and DML scripts, stored procedures, triggers, and the GUI client that connects to it.

1.2. PROBLEM STATEMENT

Public transportation authorities in Mianwali previously had no centralized digital system to manage bus operations. Passengers had no reliable way to know the real-time location of buses, the routes available, or the expected arrival time at a given stop, leading to wasted waiting time and inconvenience.

On the operational side, administrators managing the electric bus fleet faced difficulties in keeping track of driver duty schedules, bus maintenance and availability status, route and stop information, and historical operational data. Manual record-keeping resulted in data redundancy, scheduling conflicts (such as assigning one driver to two overlapping duties, or one bus to two routes at the same time), and an absence of structured reporting.

The core problem addressed by this project is therefore the **absence of a normalized, constraint-enforced relational database** capable of supporting real-time bus tracking, route and schedule management, driver duty lifecycle management, and administrative reporting, while preventing data redundancy and scheduling conflicts through database-level business rules.

1.3. PROJECT OBJECTIVES

The general objective of this project is to design, normalize, and implement a relational database for the Electric Bus Location Tracker System, and to build a GUI client that interacts with this database exclusively through stored procedures, functions, and triggers.

The specific objectives are:

- Convert the approved Entity-Relationship Diagram (ERD) from Milestone 2 into a complete relational schema expressed in Third Normal Form (3NF).

- Identify functional dependencies for every relation and document the normalization steps from 1NF through 3NF.
- Implement the complete schema – twelve base tables, one reporting view, four duty-lifecycle triggers, and twenty stored procedures – using a single `dbDDL.sql` script.
- Populate the database with representative sample data through a `dbDML.sql` script, with a minimum of 15–20 rows per table.
- Develop a Flutter-based, role-specific GUI (Admin, Driver, and Passenger interfaces) for performing data entry, retrieval, update, and deletion operations.
- Connect the GUI to the database through a Node.js/Express REST API, ensuring that every database operation is performed exclusively through stored procedures, functions, or triggers – never through direct ad-hoc SQL from the client.
- Enforce business rules such as preventing overlapping duty assignments, restricting duty status transitions, and validating account/bus status before assigning duties, through database triggers.

CHAPTER 2: CONCEPTUAL DATABASE DESIGN

2.1. ENTITY

The following entities were identified as central to the Electric Bus Location Tracker System, carried forward from the approved ERD in Milestone 2. The `route_stop_details` associative entity is excluded here as it is a bridge/associative relation rather than a core entity.

Table 1: List of Entities

Sr. No	Entity Name	Description
01	User	A registered individual in the system, acting either as a Driver or an Administrator. Stores account credentials, role, and status.
02	Driver	A subtype of User who operates an electric bus, with license and duty status information.
03	Admin	A subtype of User who manages buses, routes, drivers, schedules, duties, and reports.
04	Bus	An electric bus vehicle in the fleet, identified by registration number, capacity, model, and operational status.
05	Route	A predefined path between a starting point and a destination, including geographic coordinates and distance.
06	Stop	A bus stop location where passengers may board or alight, with geographic coordinates.
07	Schedule	A planned trip linking a Route and a Bus to a departure time, arrival time, and service date.
08	Duty_Assignment	A driver's assigned shift, linking a Driver, Bus, and Schedule, created and managed by an Administrator.
09	Bus_Location	A timestamped GPS observation recorded while a bus is on an active duty.
10	Report	An operational summary (daily, weekly, or monthly) generated by an Administrator.
11	Audit_Log	A system-level record of significant user actions, retained for traceability.

2.2. DATA DICTIONARY

This section presents the attributes of each entity, their data types, constraints, and descriptions, as implemented in `dbDDL.sql`. Foreign keys are documented separately in Chapter 3 (Relational Schema) and are not repeated here.

User

Table 2: Data Dictionary – User

Sr.	Name	Data Type	Constraint	Description
01	user_id (PK)	INT	AUTO INCREMENT, PRIMARY KEY	Unique identifier for the user
02	username	VARCHAR(50)	NOT NULL, UNIQUE	Login username
03	user_code	VARCHAR(30)	NOT NULL, UNIQUE	Human-readable user code (e.g. ADM-001)
04	name	VARCHAR(100)	NOT NULL	Full name of the user
05	email	VARCHAR(100)	NOT NULL, UNIQUE	Email address
06	contact	VARCHAR(20)	NOT NULL	Phone number
07	password_hash	VARCHAR(255)	NOT NULL	Bcrypt-hashed password
08	role	ENUM	('Driver','Admin'), NOT NULL	Subtype discriminator
09	account_status	ENUM	('Active','Inactive'), DEFAULT 'Active'	Login eligibility status
10	login_attempts	INT	NOT NULL, DEFAULT 0	Consecutive failed login count
11	lock_until	DATETIME	NULL	Account lock expiry timestamp
12	reset_code	VARCHAR(10)	NULL	Password-reset verification code
13	reset_code_expiry	DATETIME	NULL	Reset code expiry timestamp
14	creation_date	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Account creation timestamp
15	deletion_date	DATETIME	NULL	Soft-deletion timestamp



Driver

Table 3: Data Dictionary – Driver

Sr.	Name	Data Type	Constraint	Description
01	driver_id (PK)	INT	AUTO_INCREMENT, PRIMARY KEY	Unique identifier for the driver
02	license_no	VARCHAR(50)	NOT NULL, UNIQUE	Driving license number
03	hire_date	DATE	NOT NULL	Date the driver was hired
04	address	VARCHAR(255)	NULL	Residential address
05	status	ENUM	('Available', 'On-Duty', 'Off-Duty', 'On-Leave'), DEFAULT 'Available'	Current operational status

Admin

Table 4: Data Dictionary – Admin

Sr.	Name	Data Type	Constraint	Description
01	admin_id (PK)	INT	AUTO_INCREMENT, PRIMARY KEY	Unique identifier for the administrator



Bus

Table 5: Data Dictionary – Bus

Sr.	Name	Data Type	Constraint	Description
01	bus_id (PK)	INT	AUTO INCREMENT, PRIMARY KEY	Unique identifier for the bus
02	bus_number	VARCHAR(20)	NOT NULL, UNIQUE	Vehicle registration number
03	capacity	INT	NOT NULL, DEFAULT 40	Passenger seating capacity
04	model	VARCHAR(80)	NOT NULL, DEFAULT 'Electric Bus'	Bus model/description
05	status	ENUM	('Active', 'Maintenance', 'Inactive'), DEFAULT 'Active'	Operational status
06	registration_date	DATE	NOT NULL	Date the bus was registered
07	created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Record creation timestamp



Route

Table 6: Data Dictionary – Route

Sr.	Name	Data Type	Constraint	Description
01	route_id (PK)	INT	AUTO INCREMENT PRIMARY KEY	Unique identifier for the route
02	route_code	VARCHAR(30)	NOT NULL, UNIQUE	Human-readable route code
03	name	VARCHAR(100)	NOT NULL	Route name
04	starting_point	VARCHAR(100)	NOT NULL	Origin location name
05	start_latitude	DECIMAL(10,7)	NOT NULL	Origin latitude
06	start_longitude	DECIMAL(10,7)	NOT NULL	Origin longitude
07	destination_point	VARCHAR(100)	NOT NULL	Destination location name
08	destination_latitude	DECIMAL(10,7)	NOT NULL	Destination latitude
09	destination_longitude	DECIMAL(10,7)	NOT NULL	Destination longitude
10	distance	DECIMAL(10,2)	NOT NULL	Total route distance (km)
11	estimated_duration	INT	NOT NULL	Estimated travel time (minutes)
12	status	ENUM	('Active','Inactive'), DEFAULT 'Active'	Route status
13	created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Record creation timestamp

Stop

Table 7: Data Dictionary – Stop

Sr.	Name	Data Type	Constraint	Description
01	stop_id (PK)	INT	AUTO_INCREMENT, PRIMARY KEY	Unique identifier for the stop
02	stop_code	VARCHAR(30)	NOT NULL, UNIQUE	Human-readable stop code
03	name	VARCHAR(100)	NOT NULL	Stop name
04	latitude	DECIMAL(10,7)	NOT NULL	Stop latitude
05	longitude	DECIMAL(10,7)	NOT NULL	Stop longitude
06	creation_date	DATETIME	NOT NULL, DEFAULT CUR- RENT_TIMESTAMP	Record creation timestamp
07	deletion_date	DATETIME	NULL	Soft-deletion timestamp
08	status	ENUM	('Active','Inactive'), DEFAULT 'Active'	Stop status

Schedule

Table 8: Data Dictionary – Schedule

Sr.	Name	Data Type	Constraint	Description
01	schedule_id (PK)	INT	AUTO_INCREMENT, PRIMARY KEY	Unique identifier for the schedule
02	departure_time	TIME	NOT NULL	Scheduled departure time
03	arrival_time	TIME	NOT NULL	Scheduled arrival time
04	service_date	DATE	NOT NULL	Date of the scheduled trip
05	status	ENUM	('Scheduled','Completed', 'Cancelled'), DEFAULT 'Scheduled'	Trip status

Duty_Assignment

Table 9: Data Dictionary – Duty_Assignment

Sr.	Name	Data Type	Constraint	Description
01	duty_id (PK)	INT	AUTO_INCREMENT, PRIMARY KEY	Unique identifier for the duty
02	scheduled_date	DATE	NOT NULL	Date the duty is scheduled
03	scheduled_start_time	TIME	NOT NULL	Planned start time
04	scheduled_end_time	TIME	NOT NULL	Planned end time
05	actual_start_time	DATETIME	NULL	Timestamp duty actually started
06	actual_end_time	DATETIME	NULL	Timestamp duty actually completed
07	status	ENUM	('Scheduled','In-Progress','Completed','Skipped'), DEFAULT 'Scheduled'	Duty lifecycle status
08	completion_note	TEXT	NULL	Optional note recorded on completion
09	created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Record creation timestamp

Bus_Location

Table 10: Data Dictionary – Bus_Location

Sr.	Name	Data Type	Constraint	Description
01	location_id (PK)	BIGINT	AUTO_INCREMENT, PRIMARY KEY	Unique identifier for the location record
02	latitude	DECIMAL(10,7)	NOT NULL	Recorded latitude
03	longitude	DECIMAL(10,7)	NOT NULL	Recorded longitude
04	speed	DECIMAL(7,2)	NOT NULL, DEFAULT 0	Recorded speed (km/h)
05	is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Whether this is a current live observation
06	recorded_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Time the location was recorded

Report

Table 11: Data Dictionary – Report

Sr.	Name	Data Type	Constraint	Description
01	report_id (PK)	INT	AUTO_INCREMENT, PRIMARY KEY	Unique identifier for the report
02	report_code	VARCHAR(60)	NOT NULL, UNIQUE	Human-readable report code
03	type	ENUM	('daily','weekly', 'monthly'), NOT NULL	Report period type
04	period_start	DATE	NOT NULL	Start date of the reporting period
05	period_end	DATE	NOT NULL	End date of the reporting period
06	generated_at	DATETIME	NOT NULL, DEFAULT CUR- RENT_TIMESTAMP	Generation timestamp
07	total_duties	INT	NOT NULL, DEFAULT 0	Total duties in the period
08	completed_duties	INT	NOT NULL, DEFAULT 0	Completed duties in the period
09	skipped_duties	INT	NOT NULL, DEFAULT 0	Skipped duties in the period
10	total_buses	INT	NOT NULL, DEFAULT 0	Total buses recorded at generation time
11	total_drivers	INT	NOT NULL, DEFAULT 0	Total drivers recorded at generation time
12	active_drivers	INT	NOT NULL, DEFAULT 0	Active drivers at generation time
13	pdf_path	VARCHAR(255)	NULL	Path to the generated PDF file
14	description	TEXT	NULL	Free-text report summary

Audit_Log

Table 12: Data Dictionary – Audit_Log

Sr.	Name	Data Type	Constraint	Description
01	audit_id (PK)	BIGINT	AUTO_INCREMENT, PRIMARY KEY	Unique identifier for the audit record
02	action	VARCHAR(80)	NOT NULL	Action performed (e.g. LOGIN, CREATE_DUTY)
03	entity_name	VARCHAR(80)	NULL	Name of the affected entity/table
04	entity_id	VARCHAR(80)	NULL	Identifier of the affected record
05	details	TEXT	NULL	Free-text details of the action
06	created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Timestamp of the action

2.3. BUSINESS RULES

The following business rules govern the operation of the Electric Bus Location Tracker System and are enforced at the database level through constraints and triggers (`trg_duty_before_insert`, `trg_duty_after_insert`, `trg_duty_before_update`, `trg_duty_after_update`):

- A User must be either a Driver or an Admin, not both; the `role` attribute enforces this through the `users` table, with `drivers` and `admins` as 1:1 subtype tables.
- A duty's scheduled end time must always be later than its scheduled start time; otherwise the insert or update is rejected.
- A Driver cannot be assigned two overlapping duties (same date, overlapping time range) while either duty is `Scheduled` or `In-Progress`.
- A Bus cannot be assigned to two overlapping duties (same date, overlapping time range) while either duty is `Scheduled` or `In-Progress`.
- A duty can only be created for a Driver whose linked User account has `account_status = 'Active'`.
- A duty can only be created for a Bus whose `status = 'Active'`.
- A duty's status cannot transition backward from `Completed` to `Scheduled`.
- When a duty's status changes to `In-Progress`, the assigned Driver's status is automatically set to `On-Duty`; when it changes to `Completed` or `Skipped`, the Driver's status reverts to `Available`.
- A User account is locked for a defined period after five consecutive failed login attempts (`login_attempts`, `lock_until`).
- A password-reset code is valid for five minutes only (`reset_code_expiry`).
- Drivers, Admins, Buses, Routes, and Stops are never physically deleted by the

application; `deletion_date` or `status = 'Inactive'` is used instead (soft deletion), preserving historical duty and report records.

- A Route must contain at least one Stop, defined through the `route_stop_details` bridge table with a unique `stop_order` per route.
- Every Report must be linked to exactly one Admin who generated it.
- Every Bus_Location record is linked to the Bus, Driver, and Route active at the time of recording, and optionally to the Duty_Assignment in progress.

2.4. ENTITY RELATIONSHIP DIAGRAM

The ERD presented below is the approved diagram from Milestone 2, carried forward as the conceptual foundation for the relational schema in Chapter 3. It uses Chen notation: rectangles for entities, diamonds for relationships, ovals for attributes (primary keys underlined), and crow's-foot-style cardinality markers on each connecting line.

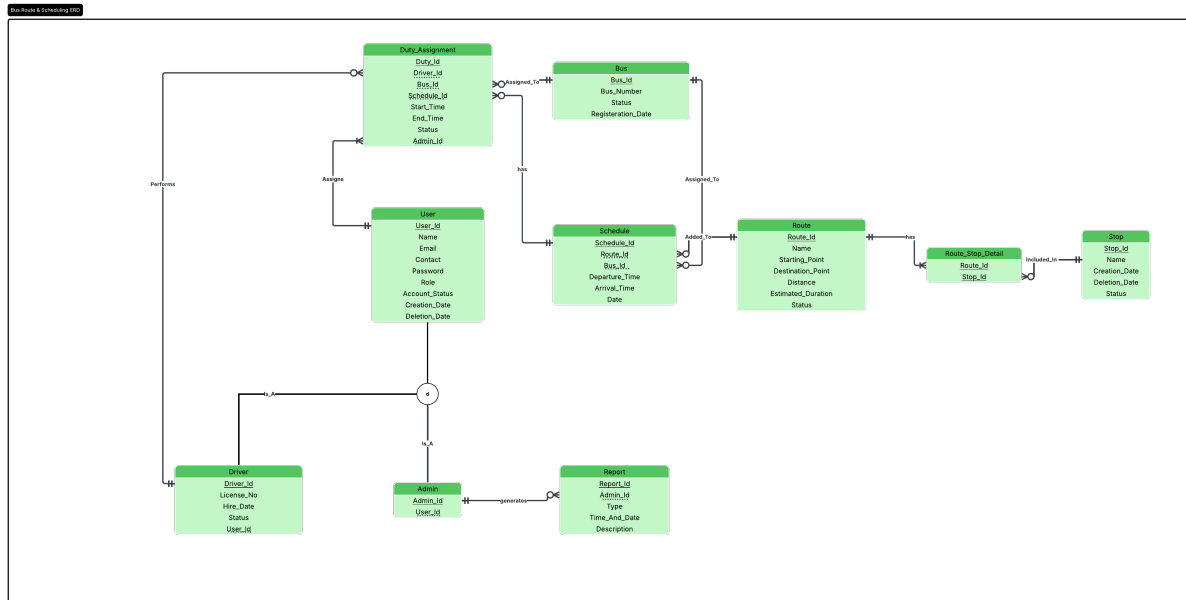


Figure 1: Entity-Relationship Diagram for the Electric Bus Location Tracker System

CHAPTER 3: LOGICAL DATABASE DESIGN

3.1. RELATIONAL SCHEMA

The ERD was converted into a relational schema following standard ER-to-relation mapping rules: each regular entity becomes a table with the entity's identifier as primary key; each 1:1 relationship (User–Driver, User–Admin) is implemented by placing a unique foreign key in the subtype table referencing the supertype; each 1:N relationship is implemented by placing a foreign key in the "many" side table; and the M:N relationship (Route–Stop) is implemented through the associative table `route_stop_details`.

The resulting twelve tables, with primary keys (PK) and foreign keys (FK), are:

1. **users** (user_id PK, username, user_code, name, email, contact, password_hash, role, account_status, login_attempts, lock_until, reset_code, reset_code_expiry, cre-

- ation_date, deletion_date)
2. **drivers** (driver_id PK, user_id FK → users, license_no, hire_date, address, status)
3. **admins** (admin_id PK, user_id FK → users)
4. **buses** (bus_id PK, bus_number, capacity, model, status, registration_date, created_at)
5. **routes** (route_id PK, route_code, name, starting_point, start_latitude, start_longitude, destination_point, destination_latitude, destination_longitude, distance, estimated_duration, status, created_at)
6. **stops** (stop_id PK, stop_code, name, latitude, longitude, creation_date, deletion_date, status)
7. **route_stop_details** (route_id, stop_id composite PK, route_id FK → routes, stop_id FK → stops, stop_order, distance_from_start, estimated_minutes_from_start)
8. **schedules** (schedule_id PK, route_id FK → routes, bus_id FK → buses, departure_time, arrival_time, service_date, status)
9. **duty_assignments** (duty_id PK, driver_id FK → drivers, bus_id FK → buses, schedule_id FK → schedules, admin_id FK → admins, scheduled_date, scheduled_start_time, scheduled_end_time, actual_start_time, actual_end_time, status, completion_note, created_at)
10. **bus_locations** (location_id PK, bus_id FK → buses, driver_id FK → drivers, route_id FK → routes, duty_id FK → duty_assignments, latitude, longitude, speed, is_active, recorded_at)
11. **reports** (report_id PK, admin_id FK → admins, report_code, type, period_start, period_end, generated_at, total_duties, completed_duties, skipped_duties, total_buses, total_drivers, active_drivers, pdf_path, description)
12. **audit_logs** (audit_id PK, user_id FK → users (nullable), action, entity_name, entity_id, details, created_at)

Bridge / associative relation: `route_stop_details` resolves the M:N relationship between `routes` and `stops`. Its composite primary key (`route_id`, `stop_id`) ensures a stop appears at most once per route, while `stop_order` (unique per route) preserves the visiting sequence and `distance_from_start` / `estimated_minutes_from_start` record cumulative distance and travel time to that stop.

Referential integrity policy: Foreign keys generally use `ON UPDATE CASCADE` so identifier changes propagate, and `ON DELETE RESTRICT` so that referenced rows (`drivers`, `buses`, `routes`, `schedules`, `admins`) cannot be removed while dependent rows exist – this is the database-level enforcement of the soft-deletion business rule. The exception is `bus_locations.duty_id`, which uses `ON DELETE SET NULL` since a location record remains meaningful even if its originating duty record is later removed, and `audit_logs.user_id`, which uses `ON DELETE SET NULL` to preserve audit history after a user record is gone.

3.2. FUNCTIONAL DEPENDENCIES

The functional dependencies (FDs) for each relation are listed below, using the notation $A \rightarrow B$ to mean attribute(s) A functionally determine attribute(s) B .



users

user_id $\rightarrow$ {username, user_code, name, email, contact, password_hash, role, account_status, login_attempts}

username $\rightarrow$ user_id email $\rightarrow$ user_id user_code $\rightarrow$ user_id

Example: user_id = 1 $\rightarrow$ (username = 'admin_main', role = 'Admin', account_status = 'Active').

drivers

driver_id $\rightarrow$ {user_id, license_no, hire_date, address, status} user_id $\rightarrow$ driver_id license_no $\rightarrow$ driver_id

admins

admin_id $\rightarrow$ user_id user_id $\rightarrow$ admin_id

buses

bus_id $\rightarrow$ {bus_number, capacity, model, status, registration_date, created_at} bus_number $\rightarrow$ bus_id

routes

route_id $\rightarrow$ {route_code, name, starting_point, start_latitude, start_longitude, destination_point, destination_latitude}

route_code $\rightarrow$ route_id

Example: route_id = 3 $\rightarrow$ (name = 'Mianwali--Kalabagh Express', distance = 28.50, estimated_duration = 45).

stops

stop_id $\rightarrow$ {stop_code, name, latitude, longitude, creation_date, deletion_date, status} stop_code $\rightarrow$ stop_id

route_stop_details

{route_id, stop_id} $\rightarrow$ {stop_order, distance_from_start, estimated_minutes_from_start}

{route_id, stop_order} $\rightarrow$ stop_id (unique ordering per route)

schedules

schedule_id $\rightarrow$ {route_id, bus_id, departure_time, arrival_time, service_date, status}

{route_id, bus_id, departure_time, service_date} $\rightarrow$ schedule_id

Example: schedule_id = 12 $\rightarrow$ (route_id = 3, bus_id = 7, departure_time = '08:00:00', service_date = '2026-06-15').

duty_assignments

duty_id $\rightarrow$ {driver_id, bus_id, schedule_id, admin_id, scheduled_date, scheduled_start_time, scheduled_end_time}

Example: duty_id = 45 $\rightarrow$ (driver_id = 5, bus_id = 7, schedule_id = 12, status = 'Completed', actual_end_time = '2026-06-15 08:47:00').

bus_locations

location_id → {bus_id, driver_id, route_id, duty_id, latitude, longitude, speed, is_active, recorded_at}

reports

report_id → {admin_id, report_code, type, period_start, period_end, generated_at, total_duties, completed_duties}

report_code → report_id

audit_logs

audit_id → {user_id, action, entity_name, entity_id, details, created_at}

3.3. NORMALIZATION

This section documents the normalization process applied to every relation, identifying any partial or transitive dependencies found during the conceptual-to-logical conversion and the steps taken to remove them.

First Normal Form (1NF)

All relations satisfy 1NF: every attribute holds a single, atomic value (e.g. `latitude` and `longitude` are stored as separate `DECIMAL(10,7)` columns rather than a combined coordinate string), and every table has a defined primary key. No repeating groups or multi-valued attributes exist in any table.

Second Normal Form (2NF)

2NF requires that no non-key attribute depend on only part of a composite primary key. Only one table in the schema has a composite primary key: `route_stop_details` (`route_id`, `stop_id`).

- **Partial dependency check:** `stop_order`, `distance_from_start`, and `estimated_minutes_from_start` all depend on the *combination* of (`route_id`, `stop_id`) – the same stop has a different order and distance on different routes – so no partial dependency exists. The table is in 2NF.
- All other tables use a single-column surrogate primary key (an auto-incrementing integer or bigint), so 2NF is trivially satisfied – there is no composite key for a non-key attribute to be partially dependent on.

Third Normal Form (3NF)

3NF requires that no non-key attribute depend transitively on the primary key through another non-key attribute. The following potential transitive dependencies were identified and resolved during conceptual-to-logical conversion:

- **users → drivers/admins:** Initially, driver-specific attributes (`license_no`, `hire_date`, `address`, `status`) and admin-specific attributes could have been stored directly in a single `users` table. This would create a transitive dependency `user_id` → `role` → `license_no` (`license_no` is only meaningful when `role` = 'Driver'), and would force NULLs for all driver columns on admin rows. This was resolved by splitting `users`

into a supertype table and two 1:1 subtype tables, **drivers** and **admins**, each with its own non-key attributes that depend only on their own primary key.

- **duty_assignments and schedules:** Route and bus information could have been duplicated directly inside **duty_assignments** (transitive dependency **duty_id** $\rightarrow$ **schedule_id** $\rightarrow$ **route_id/bus_id/departure_time/arrival_time**). Instead, **schedules** is kept as a separate relation, and **duty_assignments** references it by **schedule_id** only, while also keeping its own **driver_id**, **bus_id** (for direct duty-level queries and trigger checks), **admin_id**, and duty-specific timing/status attributes that are not derivable from the schedule alone.
- **routes and route_stop_details:** Stop names and coordinates could have been duplicated inside **route_stop_details** for convenience. Instead, **stops** remains a separate relation and **route_stop_details** stores only the foreign keys (**route_id**, **stop_id**) plus route-specific ordering/distance attributes, avoiding transitive dependency on stop attributes.
- **reports:** Aggregate counters (**total_duties**, **completed_duties**, **total_buses**, etc.) are intentionally *derived/snapshot* values computed at generation time and stored for historical auditing – they are functionally dependent only on **report_id** (the snapshot they belong to), not on any other non-key attribute of **reports**, so no transitive dependency exists.

After these transformations, every relation satisfies 1NF, 2NF, and 3NF: every non-key attribute depends on the whole primary key and only on the primary key. The complete set of 3NF relations is the twelve tables listed in Section 3.1 (Relational Schema).

CHAPTER 4: IMPLEMENTATION

4.1. LANGUAGE/FRAMEWORK

Table 13: Technology Stack

Area	Technology	Purpose
Mobile / GUI	Flutter (Dart)	Cross-platform, Android-first GUI for Admin, Driver, and Passenger roles; live maps, GPS publishing, PDF download.
Backend API	Node.js + Express 5	REST routing, JWT authentication, role-based authorization, request validation, business logic, and report generation.
Database	MySQL 8 (Amazon RDS)	Relational storage, stored procedures, functions, triggers, view, constraints, and indexes.
Authentication	JWT + bcryptjs	Seven-day session tokens and one-way password hashing.
Maps	flutter_map + OpenStreetMap + OSRM	Map rendering and road-following route geometry without a paid Maps API key.
Reports	PDFKit	Server-side generation of daily, weekly, and monthly operational PDF reports.
Email	AWS SES v2	Delivery of password-reset verification codes.
Cloud Hosting	AWS Elastic Beanstalk + RDS	Managed Node.js deployment and managed MySQL database.

Flutter was selected for the GUI because it allows a single Dart codebase to target Android (and, with no extra effort, iOS) while providing native-feeling widgets, a mature mapping ecosystem (`flutter_map`), and built-in HTTP and geolocation packages required for live GPS publishing. Node.js with Express was chosen for the API layer because of its lightweight routing, mature MySQL driver (`mysql2`) with connection pooling and promise support, and the wide availability of middleware for JWT authentication (`jsonwebtoken`), validation (`express-validator`), and security headers (`helmet`, `cors`). MySQL 8 was chosen as the DBMS because it natively supports stored procedures, triggers, views, the `ENUM` type used extensively for status fields, and is fully supported by Amazon RDS for managed cloud hosting.

4.2. DATABASE CONNECTIVITY

The Flutter GUI never connects to MySQL directly. All database access is mediated by the Node.js/Express REST API, which uses the `mysql2` driver's promise-based connection pool (`backend/src/config/database.js`) to communicate with the MySQL/RDS instance.

The data flow is as follows:

1. The Flutter app sends an HTTP request (with a JWT in the `Authorization` header for protected routes) to the configured API base URL.
2. Express middleware (`backend/src/middleware/auth.js`) verifies the JWT and checks the user's role (Admin or Driver) before allowing the request to reach the controller.
3. The relevant controller (`adminController.js`, `authController.js`, `dutyController.js`, `gpsController.js`, `reportController.js`, or `routeController.js`) validates the request body and calls the appropriate **stored procedure** through the connection pool – `CALL sp_xxx(...)`.
4. MySQL executes the stored procedure. Triggers on `duty_assignments` (`trg_duty_before_insert`, `trg_duty_after_insert`, `trg_duty_before_update`, `trg_duty_after_update`) enforce business rules and side effects automatically.
5. The procedure's result set is returned to the controller, which formats it as JSON and sends it back to the Flutter app.

Connection parameters (host, port, user, password, database name, and pool size) are read from environment variables (`backend/.env`, based on `backend/.env.example`) and are never hard-coded or committed to source control. In production, the RDS security group permits MySQL traffic on port 3306 only from the Elastic Beanstalk application's security group, not from the public internet.

Direct SQL policy: Per the milestone requirement, the backend controllers issue `CALL` statements to stored procedures for every create, read, update, and delete operation exposed to the GUI. No controller constructs and executes ad-hoc `SELECT/INSERT/UPDATE/DELETE` statements against the application tables directly.

4.3. STORED PROCEDURES AND FUNCTIONS

The database implements **20 stored procedures** and **4 triggers**, all of which are actively called by the backend controllers. The table below documents each database programmability object, its type, purpose, inputs, and outputs.

	Object Name	Type	Description	Input Parameters	Output / Return Value
1	<code>sp_get_user_for_login</code>	Proc.	Retrieves a user record by username or user code for login validation.	<code>p_username</code> , <code>p_user_code</code>	User row (id, password_hash, role, account_status, lock state)



	Object Name	Type	Description	Input Parameters	Output / Return Value
2	sp_record_login_failure	Proc.	Increments login_attempts and locks the account for a cool-down period after 5 failures.	p_user_id	None (row update)
3	sp_record_login_success	Proc.	Resets login_attempts and lock_until on successful login.	p_user_id	None (row update)
4	sp_get_user_profile	Proc.	Returns profile information (joined with driver/admin subtype) for the authenticated user.	p_user_id	Profile row
5	sp_change_password	Proc.	Updates password_hash for a logged-in user after current-password verification.	p_user_id, p_password_hash	None (row update)
6	sp_save_reset_code	Proc.	Stores a hashed reset code and 5-minute expiry for the given email.	p_email, p_reset_code	None (row update)
7	sp_reset_password	Proc.	Verifies the reset code/expiry and stores the new password hash.	p_email, p_reset_code, p_password_hash	Success/failure status
8	sp_get_routes	Proc.	Returns all active routes with their stop sequence.	None	Route rows
9	sp_create_route	Proc.	Inserts a new route with start/destination coordinates and distance/duration.	p_route_code, p_name, p_starting_point, p_start_latitude, p_start_longitude, p_destination_point, p_destination_latitude, p_destination_longitude, p_distance, p_estimated_duration	New route_id
10	sp_get_drivers	Proc.	Returns all drivers joined with their user account information.	None	Driver rows
11	sp_update_driver	Proc.	Updates a driver's profile and linked user account fields.	p_driver_id, p_name, p_email, p_contact, p_license_no, p_driver_status, p_address	None (row update)

Object Name	Type	Description	Input Parameters	Output / Return Value
12 sp_set_driver_account_status	Proc.	Activates or deactivates a driver's user account (soft enable/disable).	p_driver_id, p_account_status	None (row update)
13 sp_create_duty	Proc.	Creates a new duty assignment; validation triggers run automatically.	p_driver_id, p_bus_id, p_route_id, p_scheduled_date, p_scheduled_start_time, p_scheduled_end_time, p_admin_id	New duty_id
14 sp_get_admin_duties	Proc.	Returns all duty assignments with driver, bus, and route details for the admin dashboard.	None	Duty rows
15 sp_get_driver_monthly_duties	Proc.	Returns a driver's duties for a given month and year.	p_user_id, p_month, p_year	Duty rows
16 sp_start_duty	Proc.	Marks a duty In-Progress and records actual_start_time .	p_user_id, p_duty_id	None (row update)
17 sp_complete_duty	Proc.	Marks a duty Completed , records actual_end_time and an optional note.	p_user_id, p_duty_id, p_note	None (row update)
18 sp_update_bus_location	Proc.	Inserts a new GPS observation for a bus during an active duty.	p_user_id, p_bus_id, p_route_id, p_duty_id, p_latitude, p_longitude, p_speed	New location_id
19 sp_get_reports	Proc.	Returns all generated reports with admin details.	None	Report rows
20 sp_create_report	Proc.	Inserts a new operational report with computed statistics and PDF path.	p_admin_id, p_report_code, p_type, p_period_start, p_period_end, p_total_duties, p_completed_duties, p_skipped_duties, p_total_buses, p_total_drivers, p_active_drivers, p_pdf_path, p_description	New report_id

	Object Name	Type	Description	Input Parameters	Output / Return Value
21	trg_duty_before_insert	Trigger	Validates time ordering, driver/bus overlap, and driver/bus active status before a new duty is inserted.	(trigger – NEW row)	SIGNAL on violation
22	trg_duty_after_insert	Trigger	Sets the driver's status to On-Duty if the new duty is inserted as In-Progress .	(trigger – NEW row)	Updates drivers.status
23	trg_duty_before_update	Trigger	Validates time ordering, blocks Completed→Scheduled transitions, and re-checks driver/bus overlap on update.	(trigger – NEW/OLD row)	SIGNAL on violation
24	trg_duty_after_update	Trigger	Synchronizes the driver's status (On-Duty / Available) with the duty's new status.	(trigger – NEW row)	Updates drivers.status

In addition to the procedures and triggers above, the schema defines one reporting view, `view_admin_dashboard_stats`, which aggregates active bus count, total buses, total drivers, available drivers, today's duties, today's completed duties, and active routes into a single row for the administrator dashboard.

4.4. INTERFACES

The GUI is implemented as a Flutter mobile application with three role-specific interface groups, all communicating with the Express API described in Section 4.2.

Administrator Interfaces

- **Login and Dashboard:** Secure login screen; dashboard showing the route overview and live statistics from `view_admin_dashboard_stats`.
- **Driver Management:** Screens to register a new driver, list all drivers, edit a driver's profile, and activate/deactivate a driver account (`sp_get_drivers`, `sp_update_driver`, `sp_set_driver_account_status`).
- **Bus Management:** Screens to add, edit, and deactivate buses, showing capacity, model, and operational status.
- **Route Management:** A map-based screen to create a new route by selecting start, destination, and intermediate stops, with automatic distance and duration calculation (`sp_create_route`, `sp_get_routes`).
- **Duty Management:** Screens to create, edit, view, and soft-delete duty assignments, selecting a driver, bus, route/schedule, date, and time window (`sp_create_duty`, `sp_get_admin_duties`).

- **Reports:** A screen to generate daily/weekly/monthly PDF reports and download them to the device (`sp_create_report`, `sp_get_reports`).
- **Profile:** View profile, change password, and request a password reset via email code.

Driver Interfaces

- **Duty Views:** Screens listing current, upcoming, completed, pending, and monthly duties (`sp_get_driver_monthly_duties`).
- **Start/Complete Duty:** A screen to start an eligible duty (`sp_start_duty`), during which the app periodically calls `sp_update_bus_location` to publish GPS coordinates, and to complete the duty at the destination (`sp_complete_duty`).
- **Profile:** View profile information and change or reset the password (`sp_get_user_profile`, `sp_change_password`, `sp_reset_password`).

Passenger / Public Interfaces

- **Route and Stop Search:** A search screen that queries active routes and stops and moves the map camera to the selected result (`sp_get_routes`).
- **Live Map:** A map screen showing the active route path, direction arrows, live bus positions (sourced from `bus_locations` via the GPS endpoints), driver/bus details, and stop ETAs.
- **Favourites:** Locally stored favourite routes/stops on the device (no passenger account or table required).

TASK DIVISION

Table 15: Task Distribution for Milestone 2

Team Member	Role	Responsibilities
Ali Abbas (NUM-BSCS-2024-06)	Database Design	Identify entities and attributes, define all functional relationships and cardinalities, design ERD using Crow's Foot notation and review the final app functionalities
Laiba Taj (NUM-BSCS-2024-31)	Documentation and Testing Queries	Creates the Dbdml.sql file, review Dbddl.sql file, done testing and manages repository
Mehroz Ali Khan (NUM-BSCS-2024-34)	Implementation	Creates Dbddl.sql and created the actual working application

REFERENCES

1. Chen, P. P., "The Entity-Relationship Model—Toward a Unified View of Data," *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, 1976.

2. R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.
3. MySQL 8.0 Reference Manual, Oracle Corporation, 2024. [Online]. Available: <https://dev.mysql.com/doc/>
4. Express.js Documentation. [Online]. Available: <https://expressjs.com/>
5. Flutter Documentation, Google. [Online]. Available: <https://flutter.dev/docs>
6. Node.js Documentation. [Online]. Available: <https://nodejs.org/docs>
7. OpenStreetMap and OSRM Project Documentation. [Online]. Available: <https://project-osrm.org/>
8. Electric Bus Location Tracker – Software Requirements Specification, Namal University, Mianwali, 2024.
9. Electric Bus Location Tracker – ERD and Conceptual Database Design (CCP Milestone 2), Namal University, Mianwali, 2026.
10. GitHub Repository Source Code: <https://github.com/MAKDevelopers34/App-Development/tree/main/electric-bus-tracker>
11. GitHub Repository Material and Docx: https://github.com/LaibaTaj31/Electric_Buses_Tracker_DB-project.git
12. Drive link to download Apk: https://drive.google.com/drive/folders/1NvnH2_X3bGsIEtyZSMLtdhTLDX6DaCJi?usp=sharing

APPENDIX: AI USAGE AND PROMPTS

Table 16: AI Tools Usage

Tool Used	Purpose	Date Accessed
ChatGPT (OpenAI)	Initial guidance on ERD-to-relational mapping and stored procedure structure.	March–May 2026
Claude (Anthropic)	ERD validation (Milestone 2), conversion of the approved ERD to a 3NF relational schema, functional dependency analysis, normalization documentation, and drafting of this Milestone 3 report from the implemented dbDDL.sql/dbDML.sql scripts and final technical reference document.	March–June 2026

Prompts Used

1. “Based on this SRS for an electric bus tracking system, help me identify all entities needed for the database design.”
2. “Validate my supertype-subtype structure for User with Driver and Admin subtypes. What are the correct cardinalities?”
3. “Convert the approved ERD from Milestone 2 into a 3NF relational schema, listing primary keys, foreign keys, and the bridge table for the Route–Stop relationship.”
4. “Given these table definitions from dbDDL.sql, identify the functional dependencies

for each table and document the normalization steps from 1NF to 3NF, including any transitive dependencies that were resolved.”

5. “Document the 20 stored procedures and 4 triggers in dbDDL.sql in a table with object name, type, description, input parameters, and output, following the provided Database Design Document template.”
6. “Generate the Milestone 3 report in LaTeX following the provided Database Design Document template structure (chapters, headings, formatting), using the final technical report and dbDDL.sql/dbDML.sql content as source material.”

AI Usage Statement

AI tools were used for guidance, validation, normalization analysis, and formatting/documentation assistance based on the team’s own implemented database scripts (`dbDDL.sql`, `dbDML.sql`) and source code. All schema decisions, business rules, and final content were reviewed and approved by the team.